Ahora reviso los módulos principales (scraper y tts, que son los que están "funcionales").He revisado el proyecto completo (`main.py`, `config.py`, los 4 módulos, README, requirements y `.gitignore`). Esto es lo que encontré:

1. La documentación miente sobre el estado real del código (bug importante) El README y los comentarios de `main.py` dicen que los Módulos 3 (`video_processor.py`) y 4 (`video_editor.py`) son "placeholders para la Fase 2". Pero no lo son: están completamente implementados, con lógica real de selección de clip, transcripción con Whisper, recorte a 9:16 y renderizado. Consecuencia práctica: en `main.py` hay bloques `except NotImplementedError` que nunca van a saltar, porque esas funciones no lanzan `NotImplementedError` sino `RuntimeError`, `FileNotFoundError`, `ValueError` o excepciones de MoviePy/Whisper. Si algo falla ahí (por ejemplo, no hay clips de gameplay suficientemente largos), el script no mostrará el mensaje "⚠️ Módulo aún no implementado" — va a reventar con un traceback sin capturar.

2. Importaciones pesadas incondicionales `main.py` importa `whisper`, `torch` y `moviepy` (vía `video_editor` y `video_processor`) al principio del archivo, aunque el README sugiere que solo se necesitan para la "Fase 2". Si alguien solo instala las dependencias de scraping+TTS, el script falla al importar, antes incluso de intentar el scraping.

3. `requirements.txt` sin techo de versión para MoviePy `moviepy>=1.0.3` no pone límite superior. `video_editor.py` usa la API de MoviePy 1.x (`moviepy.editor`, `moviepy.video.fx.all`, `TextClip(fontsize=...)`, `.set_start()`, `.set_audio()`...), que cambió drásticamente en MoviePy 2.x. Un `pip install -r requirements.txt` hoy podría instalar la v2 y romper todo el Módulo 4.

4. Falta ImageMagick como prerequisito `TextClip` necesita ImageMagick instalado en el sistema. Hay un comentario suelto en el código que lo menciona, pero no está en el README ni en requirements — nadie que siga las instrucciones de instalación se va a enterar hasta que falle.

5. Mezcla de idiomas subreddit/voz `SUBREDDITS = ["AmItheAsshole", "HistoriasDeReddit", "confesiones"]` mezcla un subreddit en inglés con dos en español, pero `TTS_VOICE` está fijada a una sola voz en español (`es-ES-AlvaroNeural`). Si el scraper coge una historia de AmItheAsshole, terminarás con texto en inglés narrado con voz española — resultado inservible.

6. Un post se marca "usado" antes de confirmar que el pipeline tuvo éxito En `scraper.py`, `_save_used_post(post.id)` se llama en el momento en que se encuentra la historia, no después de que el audio/vídeo se generen correctamente. Si el TTS o el montaje fallan después, ese post queda "quemado" para siempre aunque nunca se haya producido nada con él.

7. `video_editor.py` no captura ninguna excepción A diferencia de `scraper.py` y `video_processor.py`, que sí envuelven sus operaciones críticas en `try/except`, `create_final_video()` no tiene ningún manejo de errores. Cualquier fallo de Whisper, ffmpeg o de escritura en disco tumba el proceso entero.

8. Problema de rendimiento: un `TextClip` por cada palabra Para historias de 500-1000 caracteres, esto puede ser fácilmente 150-200 palabras, cada una generando su propio `TextClip` (que internamente llama a ImageMagick). Es lento y consume mucha memoria; normalmente esto se optimiza agrupando palabras en clips reutilizables o generando los subtítulos en una sola pasada.

9. Resto de copy-paste En `video_editor.py` hay un comentario con un residuo `[cite: 9]` (`# Generar nombre de salida basado en el ID de la historia[cite: 9]`) — parece una cita que se copió sin limpiar de alguna respuesta generada por IA. No rompe nada, pero es señal de que ese archivo no se revisó línea por línea.

10. Edge case sin proteger en el cálculo de `start_time` `max_start_time = bg_clip.duration - audio.duration` seguido de `random.uniform(0, max_start_time)`: si por diferencias de precisión entre cómo `video_processor.py` mide la duración y cómo la vuelve a medir `video_editor.py`, `max_start_time` puede acabar siendo ≤ 0, y no hay ninguna comprobación antes de llamar a `random.uniform`.

Lo más urgente de arreglar sería el punto 1 (la discrepancia placeholder/implementado, que deja el manejo de errores roto) y el punto 3 (fijar la versión de MoviePy) antes de que esto muerda a alguien en producción.

esto es lo que quiero hacer: crear un plan de desarrollo paso a paso y proporcionarme el código base inicial en Python para cada módulo. El flujo de trabajo que el programa debe seguir es: Módulo 1: Scraping de Reddit: Usar PRAW para extraer la historia "Top" del día de subreddits como r/AITA o r/HistoriasDeReddit. Necesito el título y el cuerpo del texto. Filtra por longitud (ej. entre 500 y 1000 caracteres). Módulo 2: Texto a Voz (TTS): Usar la librería edge-tts (gratuita) para convertir el título y el texto en archivos de audio MP3 con una voz española natural. Módulo 3: Procesamiento de Fondo: Elegir un clip de gameplay aleatorio de una carpeta local (ej. de 10 minutos de Minecraft o GTA V que yo ya tenga), y recortar un segmento aleatorio con la misma duración exacta que el audio generado. Módulo 4: Montaje de Video: Usar MoviePy para superponer el audio sobre el video de fondo, ajustar el video a formato vertical (1080x1920) y agregar subtítulos palabra por palabra (estilo karaoke) usando Whisper (de OpenAI, localmente si es posible) para generar los tiempos de las palabras. Módulo 5: Orquestador: Un script principal (main.py) que ejecute todo el proceso secuencialmente. y esto es lo que llevo: Actualizar el código: Borra el contenido de `video_editor_2.py` y `video_processor.py`. Pega en ellos los bloques de código con la lógica de MoviePy y Whisper que te proporcioné anteriormente. PY Voy a montar el proyecto completo (estructura + Módulos 1, 2 y el orquestador) y te dejo la explicación de cada punto que pediste.

Estructuró módulos faltantes para completar arquitectura del proyecto

Estructuró módulos faltantes para completar arquitectura del proyecto

Crear la estructura de carpetas del proyecto

Crear config.py con la configuración central del proyecto

Crear __init__.py del paquete modules

Crear Módulo 1: scraper.py (extracción de historias de Reddit con PRAW)

Crear Módulo 2: tts.py (texto a voz con edge-tts)

Creating placeholder stubs for the remaining modules.

Crear placeholder del Módulo 3 (fondo de vídeo) para que main.py no rompa al importarlo

Crear placeholder del Módulo 4 (montaje de vídeo) para que main.py no rompa al importarlo

Crear el orquestador principal main.py (Módulo 5)

Listo

Ahora los archivos de configuración y documentación:

Se crearon 4 archivos

Se crearon 4 archivos

Crear requirements.txt

Crear plantilla .env.example para las credenciales

Crear .gitignore

Crear README con instrucciones de instalación y uso

Listo

Verifico que todo compila correctamente antes de entregarlo:

Estructuró explicación modular cubriendo librerías, carpetas, implementación y configuración

Estructuró explicación modular cubriendo librerías, carpetas, implementación y configuración

Aquí tienes el proyecto completo en reddit_video_bot.zip, con los Módulos 1, 2 y el orquestador ya funcionales, y los Módulos 3/4 como placeholders (lanzan NotImplementedError de forma controlada) para que main.py no rompa mientras seguimos desarrollando.

A. Librerías a instalar

bash

pip install praw edge-tts python-dotenv# Para la Fase 2 (ya incluidas en requirements.txt, no hace falta instalarlas todavía):pip install moviepy openai-whisper torch ffmpeg-python

También necesitarás ffmpeg instalado a nivel de sistema (no es paquete pip): sudo apt install ffmpeg en Linux, brew install ffmpeg en Mac, o el binario oficial en Windows. MoviePy y Whisper dependen de él.

B. Estructura de carpetas

reddit_video_bot/

├── main.py # Orquestador (Módulo 5)

├── config.py # Credenciales, rutas y constantes centralizadas

├── requirements.txt

├── .env.example

├── .gitignore

├── modules/

│ ├── scraper.py # Módulo 1 ✅

│ ├── tts.py # Módulo 2 ✅

```
│   ├── video_processor.py   # Módulo 3 ⏳ (placeholder)
│   └── video_editor.py      # Módulo 4 ⏳ (placeholder)
├── assets/
│   ├── audio/               # MP3 generados
│   ├── gameplay/            # ⚠️ copia aquí tus clips de Minecraft/GTA
│   └── output/              # Vídeos finales (Fase 2)
└── data/
    └── used_posts.json      # Histórico para no repetir historias
```

La idea de config.py es que ningún módulo tenga valores "hardcodeados" — todo (voz, rango de longitud, subreddits, rutas) se ajusta en un solo sitio.

C. Qué hace el código incluido

modules/scraper.py: recorre SUBREDDITS en orden aleatorio, mira el "top del día" de cada uno, descarta posts fijados o sin texto, filtra por longitud (500–1000 caracteres), y guarda un histórico en data/used_posts.json para no repetir historias entre ejecuciones.

modules/tts.py: usa edge_tts.Communicate (asíncrono) para narrar "título + pausa + cuerpo" en un solo MP3. Incluye --list-voices para ver todas las voces en español disponibles y elegir la que más te guste.

main.py: encadena 1 → 2, y ya deja preparadas las llamadas a los Módulos 3 y 4 (que ahora solo imprimen un aviso y se detienen, sin romper el flujo).

Pruébalo así:

bash

```
python modules/scraper.py     # solo scrapingpython modules/tts.py        # solo TTSpython main.py                # pipeline completo (Fase 1)
```

D. Configurar las credenciales de Reddit

Entra a https://www.reddit.com/prefs/apps con tu cuenta de Reddit.

Clic en "create app" (o "create another app") al final de la página.

Rellena:name: cualquier nombre, ej. RedditStoryBot

Tipo: elige script (es el que necesitas para PRAW con usuario/contraseña de app, uso personal/no interactivo)

redirect uri: pon algo como http://localhost:8080 (no se usará realmente, pero el campo es obligatorio)

Al crear la app verás:client_id: la cadena corta justo debajo del nombre de la app

client_secret: la cadena etiquetada como "secret"

Copia .env.example a .env y rellena:

```
REDDIT_CLIENT_ID=tu_client_id

REDDIT_CLIENT_SECRET=tu_client_secret

REDDIT_USER_AGENT=RedditStoryBot/1.0 by tu_usuario_de_reddit
```

El user_agent debe ser único y descriptivo — Reddit puede limitarte si usas un user-agent genérico o el de otra app.

6. .env ya está en .gitignore, así que no se subirá por accidente si usas git.

Dos avisos prácticos para cuando pases a producción:

Reddit tiene términos de uso específicos para su API (incluyendo límites/tarifas para uso a gran volumen o comercial); si el bot empieza a monetizar en TikTok/YouTube conviene revisar la política vigente de la Reddit Data API.

Considera añadir siempre una referencia al subreddit/post original en la descripción del vídeo — además de ser buena práctica, mucha audiencia de "Reddit stories" lo espera.

Cuando confirmes que el scraping y el audio funcionan bien en tu entorno, seguimos con el Módulo 3 (recorte de gameplay) y el Módulo 4 (montaje + subtítulos karaoke con Whisper), que es donde entra la parte más pesada de MoviePy.Voy a montar el proyecto completo (estructura + Módulos 1, 2 y el orquestador) y te dejo la explicación de cada punto que pediste.

Estructuró módulos faltantes para completar arquitectura del proyecto

Estructuró módulos faltantes para completar arquitectura del proyecto

Crear la estructura de carpetas del proyecto

Crear config.py con la configuración central del proyecto

Crear __init__.py del paquete modules

Crear Módulo 1: scraper.py (extracción de historias de Reddit con PRAW)

Crear Módulo 2: tts.py (texto a voz con edge-tts)

Creating placeholder stubs for the remaining modules.

Crear placeholder del Módulo 3 (fondo de vídeo) para que main.py no rompa al importarlo

Crear placeholder del Módulo 4 (montaje de vídeo) para que main.py no rompa al importarlo

Crear el orquestador principal main.py (Módulo 5)

Listo

Ahora los archivos de configuración y documentación:

Se crearon 4 archivos

Se crearon 4 archivos

Crear requirements.txt

Crear plantilla .env.example para las credenciales

Crear .gitignore

Crear README con instrucciones de instalación y uso

Listo

Verifico que todo compila correctamente antes de entregarlo:

Estructuró explicación modular cubriendo librerías, carpetas, implementación y configuración

Estructuró explicación modular cubriendo librerías, carpetas, implementación y configuración

Aquí tienes el proyecto completo en reddit_video_bot.zip, con los Módulos 1, 2 y el orquestador ya funcionales, y los Módulos 3/4 como placeholders (lanzan NotImplementedError de forma controlada) para que main.py no rompa mientras seguimos desarrollando.

A. Librerías a instalar

bash

pip install praw edge-tts python-dotenv# Para la Fase 2 (ya incluidas en requirements.txt, no hace falta instalarlas todavía):pip install moviepy openai-whisper torch ffmpeg-python

También necesitarás ffmpeg instalado a nivel de sistema (no es paquete pip): sudo apt install ffmpeg en Linux, brew install ffmpeg en Mac, o el binario oficial en Windows. MoviePy y Whisper dependen de él.

B. Estructura de carpetas

reddit_video_bot/

├── main.py # Orquestador (Módulo 5)

├── config.py # Credenciales, rutas y constantes centralizadas

├── requirements.txt

├── .env.example

├── .gitignore

├── modules/

│ ├── scraper.py # Módulo 1 ✅

│ ├── tts.py # Módulo 2 ✅

│ ├── video_processor.py # Módulo 3 ⏳ (placeholder)

│ └── video_editor.py # Módulo 4 ⏳ (placeholder)

├── assets/

```
│   ├── audio/              # MP3 generados
│   ├── gameplay/           # ⚠️ copia aquí tus clips de Minecraft/GTA
│   └── output/             # Vídeos finales (Fase 2)
└── data/
    └── used_posts.json     # Histórico para no repetir historias
```

La idea de config.py es que ningún módulo tenga valores "hardcodeados" — todo (voz, rango de longitud, subreddits, rutas) se ajusta en un solo sitio.

C. Qué hace el código incluido

modules/scraper.py: recorre SUBREDDITS en orden aleatorio, mira el "top del día" de cada uno, descarta posts fijados o sin texto, filtra por longitud (500–1000 caracteres), y guarda un histórico en data/used_posts.json para no repetir historias entre ejecuciones.

modules/tts.py: usa edge_tts.Communicate (asíncrono) para narrar "título + pausa + cuerpo" en un solo MP3. Incluye --list-voices para ver todas las voces en español disponibles y elegir la que más te guste.

main.py: encadena 1 → 2, y ya deja preparadas las llamadas a los Módulos 3 y 4 (que ahora solo imprimen un aviso y se detienen, sin romper el flujo).

Pruébalo así:

bash

```
python modules/scraper.py     # solo scrapingpython modules/tts.py         # solo TTSpython main.py                # pipeline completo (Fase 1)
```

D. Configurar las credenciales de Reddit

Entra a https://www.reddit.com/prefs/apps con tu cuenta de Reddit.

Clic en "create app" (o "create another app") al final de la página.

Rellena:name: cualquier nombre, ej. RedditStoryBot

Tipo: elige script (es el que necesitas para PRAW con usuario/contraseña de app, uso personal/no interactivo)

redirect uri: pon algo como http://localhost:8080 (no se usará realmente, pero el campo es obligatorio)

Al crear la app verás:client_id: la cadena corta justo debajo del nombre de la app

client_secret: la cadena etiquetada como "secret"

Copia .env.example a .env y rellena:

```
REDDIT_CLIENT_ID=tu_client_id
REDDIT_CLIENT_SECRET=tu_client_secret
REDDIT_USER_AGENT=RedditStoryBot/1.0 by tu_usuario_de_reddit
```

El user_agent debe ser único y descriptivo — Reddit puede limitarte si usas un user-agent genérico o el de otra app.

6. .env ya está en .gitignore, así que no se subirá por accidente si usas git.

Dos avisos prácticos para cuando pases a producción:

Reddit tiene términos de uso específicos para su API (incluyendo límites/tarifas para uso a gran volumen o comercial); si el bot empieza a monetizar en TikTok/YouTube conviene revisar la política vigente de la Reddit Data API.

Considera añadir siempre una referencia al subreddit/post original en la descripción del vídeo — además de ser buena práctica, mucha audiencia de "Reddit stories" lo espera.

Cuando confirmes que el scraping y el audio funcionan bien en tu entorno, seguimos con el Módulo 3 (recorte de gameplay) y el Módulo 4 (montaje + subtítulos karaoke con Whisper), que es donde entra la parte más pesada de MoviePy.

esto es lo que llevo hecho quiero que me digas que me falta y porque

- Añadir material rodante: Descarga algunos clips largos de Minecraft, GTA V o Parkour y guárdalos dentro de tu carpeta `assets/gameplay/`. Sin ellos, el bot no tendrá vídeos para recortar.
- Instalar librerías de Python (Fase 2): Abre tu terminal y ejecuta la instalación de las herramientas de vídeo listadas en tu `requirements.txt`: `pip install moviepy openai-whisper torch ffmpeg-python`.

Voy a aclarar primero un par de cosas que note comparando tu mensaje con lo que hay realmente en el proyecto, y luego te dejo el plan y el código.

Lo que encontré: discrepancias antes de seguir

1. **`video_editor_2.py` no existe.** En el proyecto solo hay `video_editor.py`. Puede que sea un renombrado que se quedó a medias — revísalo antes de pegar nada ahí.
2. **Los Módulos 3 y 4 ya NO son placeholders** (aunque el documento que pegaste dice que sí lanzan `NotImplementedError`). El código que hay ahora mismo en disco ya tiene implementación real de MoviePy/Whisper — es decir, en algún momento entre esa conversación y ahora alguien (¿tú, en otra sesión?) ya escribió la lógica real. Esto es importante porque significa que **ya vas más avanzado de lo que crees**, pero también que arrastras los bugs que te señalé antes (el `try/except NotImplementedError` en `main.py` nunca se dispara, no se protege el post como "usado" antes de tiempo, mezcla de idioma en subreddits, etc.).

Dado esto, en vez de "empezar desde cero", te doy una base de código **corregida** que soluciona esos problemas y sigue exactamente el flujo que describes (incluyendo que el Módulo 3 recorte el segmento él mismo, tal y como pediste, en vez de dejarlo para el Módulo 4).

Plan de desarrollo paso a paso

1. **Entorno**: `pip install -r requirements.txt` (versión corregida más abajo) + instalar `ffmpeg` e **ImageMagick** a nivel de sistema (esto último no está en ningún pip, y es la causa nº1 de que `TextClip` falle en Windows).
2. **Fase 1 aislada**: probar `python modules/scraper.py` y `python modules/tts.py` sueltos, con historias reales, hasta que ambos funcionen sin sorpresas.
3. **Material rodante**: meter 2-3 clips largos (Minecraft/GTA/Parkour) en `assets/gameplay/`.
4. **Fase 2 aislada**: probar `modules/video_processor.py` solo (dale un mp3 cualquiera de `assets/audio/` y comprueba que te devuelve un clip recortado en `assets/gameplay/tmp_cuts/`).
5. **Fase 2 aislada, parte 2**: probar `modules/video_editor.py` con el mp3 + el clip recortado del paso anterior, y confirmar que el vídeo final tiene subtítulos sincronizados.
6. **Integración**: `python main.py` de principio a fin, revisando que si algo falla a mitad, el post **no** se marca como usado (para poder reintentar).
7. **Pulido**: ajustar voces, tiempos de pausa, rango de longitud, y — cuando ya funcione — mirar rendimiento (el karaoke palabra por palabra puede ir lento en historias largas; optimizarlo es tarea para más adelante, no bloqueante ahora).

Voy a generar los archivos:Ahora el orquestador, con imports diferidos y la lógica corregida de cuándo marcar una historia como "usada":